ActiveVisionBenchmark: Stimulus Construction for Visual-Flow Orientation Experiments

Laurent Perrinet
Institut de Neurosciences de la Timone (INT),
CNRS / Aix-Marseille Université, France
laurent.perrinet@univ-amu.fr

June 29, 2026

Abstract

We introduce ActiveVisionBenchmark, a single-file Python library for constructing visual stimuli and benchmarking active vision algorithms. The primary purpose of the library is to generate oriented visual-flow stimuli: a configurable visual-flow field defines local motion direction, oriented Gabor elements are placed according to that flow, and the result is composited on a controlled-noise background with explicit random seeds for reproducibility. The reference stimulus design is documented at Orienting yourself in the visual flow. In addition, ActiveVisionBenchmark provides (i) configurable stimulus generators including Gabor patches and visual-search arrays, (ii) a minimal model interface that decouples the benchmark harness from algorithm design, (iii) a foveation operator that simulates eccentricity-dependent resolution loss, and (iv) standard signal-detection metrics—accuracy, hit rate, false-alarm rate, and d'—as well as the scanpath-level metric of search efficiency. Two reference models are included: a uniform random baseline and a contrast-saliency model with inhibition of return. The library has a single mandatory dependency (`numpy`) and optional dependencies on `scipy` and `matplotlib`. The entire implementation fits in one `.py` file to maximise portability and ease of embedding in existing workflows.

Keywords: active vision, visual search, saccades, benchmark, Python, foveation, signal-detection theory, visual flow, orientation.

1 Introduction

ActiveVisionBenchmark is a compact Python library whose primary purpose is to construct visual stimuli, with a specific emphasis on oriented visual-flow patterns. Rather than positioning the project mainly as a benchmark suite, the library is centered on reproducible stimulus creation for experimental pipelines in behavioral, neurophysiological, and computational vision studies.

Active vision—the paradigm in which a perceiving agent controls its own sensory input by moving its eyes, head, or attention—has been a central concept in computational

and biological vision research (**???**). Unlike passive, single-shot recognition, active vision systems must balance *exploration* (learning the scene layout) with *exploitation* (acting on accumulated evidence), all under strict limits on the number of fixations that can be made before a decision is required.

Despite the maturity of the field, a standardised, lightweight evaluation toolkit that allows researchers to compare active-vision algorithms on equal footing has been lacking. Existing benchmarks are either tightly coupled to specific datasets (**??**) or require heavy software stacks that make reproducibility across labs difficult.

ActiveVisionBenchmark fills this gap by providing a minimal yet complete harness that can be dropped into any Python environment with a single file copy. Its design emphasises:

- Portability. A single `.py` file with one mandatory dependency (`numpy`).
- Extensibility. A clean abstract-base-class interface (`ActiveVisionModel`) that makes it trivial to plug in custom algorithms.
- Reproducibility. All stochastic components accept a `seed` argument and use `numpy.random.Generator`.
- Evaluability. Standard signal-detection theory (SDT) metrics are computed automatically from trial results.

The remainder of this paper is organised as follows. Section **??** reviews relevant background. Section **??** describes the visual-flow stimulus definition. Section **??** describes the library design and API. Section **??** presents experiments comparing the two bundled models. Section **??** concludes with directions for future work.

2 Background

2.1 Active Vision and Saccadic Search

Human vision is acutely non-uniform: the fovea, covering approximately 2ř of visual angle, provides high-resolution colour information while the periphery offers rapidly decreasing acuity with eccentricity (**?**). To compensate, humans make 3–5 saccadic eye movements per second, redirecting the fovea to informative regions (**?**). A computational active vision system mimics this by selecting a sequence of fixation locations that maximises information gain within a budget of N fixations.

2.2 Visual Search

The classical visual-search paradigm (**?**) asks observers to detect a target item among distractors. The *set size* (number of items) and target salience jointly determine search efficiency, measured as the slope of the reaction-time vs. set-size function. ActiveVision-Benchmark operationalises this as the ratio of expected fixations under random search to observed fixations under the model.

2.3 Signal Detection Theory

Signal detection theory (**?**) separates *sensitivity* (d') from *criterion* by decomposing decisions into hits (correct target-present responses) and false alarms (incorrect target-present responses on absent trials). ActiveVisionBenchmark reports both rates and computes d' using the loglinear correction (**?**) to handle boundary cases.

3 Stimulus Definition

The implemented visual-flow stimulus is based on three principles:

1. A visual-flow field defines local motion direction around a configurable focus of expansion.
2. Local oriented elements are generated from that flow direction, with a configurable angular offset.
3. The final stimulus is composited on a controlled-noise background with explicit random seeds for reproducibility.

This formulation makes it straightforward to manipulate interpretable variables (density, orientation offset, flow origin, contrast) while preserving a compact single-file implementation. The reference stimulus design is documented in Orienting yourself in the visual flow.

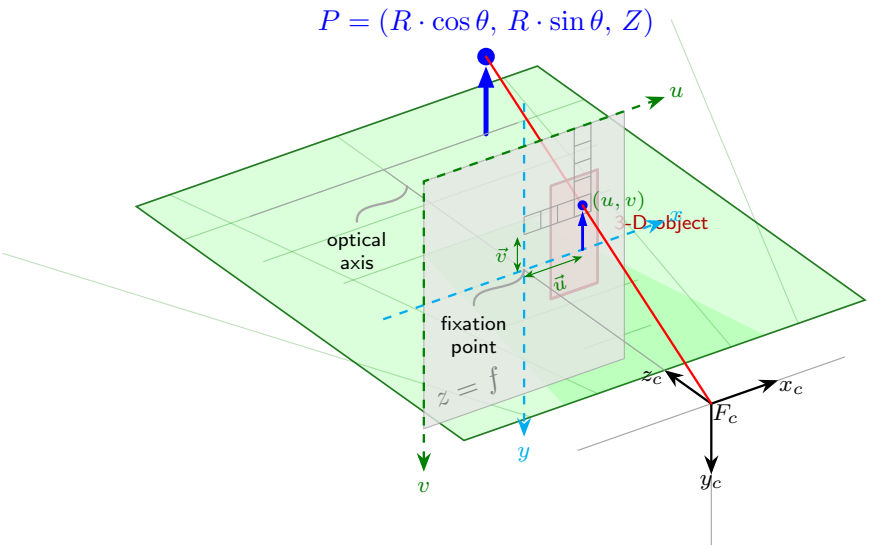


Figure 1: Schematic of the two benchmark scenario types rendered from a pinhole camera centred at F_c with focal length f. The **green ground plane** ($z = 0$) represents the *flat-scene* scenario in which targets are located on a planar surface (e.g. visual-search in a fronto-parallel stimulus). The **red cylinder** represents the *3-D object* scenario in which the agent must localise a volumetric object at polar coordinates $P = (R\cos\theta,\ R\sin\theta,\ Z)$. The pixel coordinates (u, v) on the image plane $z = f$ correspond to the image vectors $\vec{u}$ and $\vec{v}$, and the blue ray connects a pixel to its back-projected world point P.

4 Library Design

4.1 Architecture Overview

ActiveVisionBenchmark is organised around four conceptual layers, all housed in the single file `ActiveVisionBenchmark.py`:

1. Stimulus layer. Functions that generate synthetic visual stimuli.
2. Task layer. The `VisualSearchTask` class, which composes stimuli into `Trial` objects.
3. Model layer. The abstract `ActiveVisionModel` interface and two concrete implementations.
4. Benchmark layer. The `Benchmark` engine and the `BenchmarkResult` container.

Figure **??** illustrates the interaction between layers.

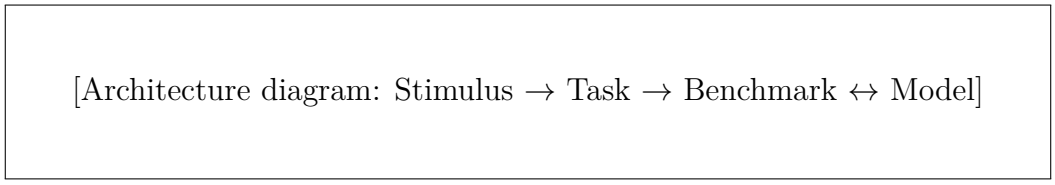


Figure 2: Schematic architecture of ActiveVisionBenchmark. Arrows indicate data flow during a benchmark run.

4.2 Stimulus Generation

The library includes low-level generators (noise and Gabor primitives), a dedicated visual-flow stimulus constructor, and optional benchmarking utilities retained for downstream model-comparison workflows. Two primitive generators are provided:

`make_gaussian_noise`. Returns a 2-D array of Gaussian luminance noise with configurable mean and standard deviation, useful as a neutral background.

`make_gabor_patch`. Generates a Gabor patch (sinusoidal grating modulated by a Gaussian envelope) parametrised by spatial frequency f, orientation θ, phase ϕ, and envelope width σ:

$$G(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \cos(2\pi f\, x_r + \phi)\,, \quad x_r = x\cos\theta + y\sin\theta. \tag{1}$$

`make_search_array`. Composes a search array by pasting distractors and an optional target onto a Gaussian-noise canvas. It returns both the image and the target location (or `None` for target-absent trials), making it suitable for supervised evaluation.

4.3 Task and Trial

`VisualSearchTask` wraps the stimulus layer to provide a reproducible stream of `Trial` objects. Each trial has a `target_present` flag and, when applicable, the ground-truth `target_location`. The `generate_dataset` method produces a list of N trials, enabling batch evaluation.

4.4 Model Interface

The abstract base class `ActiveVisionModel` defines three methods:

`reset(image)` Initialise model state for a new trial.

`next_fixation(foveated_image, fixation_history)` Given the current foveated view and the list of past fixations, return the next fixation location (r, c).

`make_decision()` Return a Boolean target-present decision at the end of the fixation budget.

This interface cleanly separates the *what to look at* (the scanning policy) from the *is the target present* (the detection decision), allowing users to mix and match components.

4.4.1 RandomSearchModel

The random baseline selects fixation locations uniformly at random over the image and decides target presence by a fair coin flip. Its expected accuracy is 50%, providing a lower bound for detection performance.

4.4.2 SaliencySearchModel

The saliency model computes local RMS contrast using a sliding window:

$$S(x, y) = \sqrt{\mathbb{E}_W[I^2] - \mathbb{E}_W[I]^2}, \tag{2}$$

where $\mathbb{E}_W[\cdot]$ denotes averaging over a square window W centred at (x, y). Fixations are drawn greedily from the maximum of S with *inhibition of return* (IoR): a circular region of radius r_{IoR} around each visited location is zeroed before the next fixation is selected.

4.5 Foveation Operator

The `foveate` function simulates foveal processing by blending the original image (inside a circular foveal region of radius r_f) with a Gaussian-blurred version (outside):

$$F(x, y) = M(x, y)\, I(x, y) + \Big(1 - M(x, y)\Big)\, I * G_{\sigma_p}(x, y), \tag{3}$$

where $M(x, y) = \mathbf{1}[\|(x, y) - (r, c)\| \le r_f]$ is the foveal mask and G_{σ_p} is a Gaussian kernel with periphery blur sigma σ_p.

4.6 Benchmark Engine

`Benchmark.run(model, trials)` iterates over trials, calling `model.reset`, then `model.next_fixation` up to `max_fixations` times, followed by `model.make_decision`. Wall-clock time is recorded per trial. The `compare` method evaluates multiple models on identical trials, ensuring a fair comparison.

4.7 Metrics

`BenchmarkResult` exposes the following scalar metrics:

Table 1: Metrics computed by `BenchmarkResult`.

Metric	Symbol	Definition
Accuracy	A	$(\text{hits} + \text{correct rejections})/N$
Hit rate	H	hits/N_+
False alarm rate	F	$\text{false alarms}/N_-$
Sensitivity	d'	$z(H) - z(F)$ (loglinear correction)
Mean fixations	$\bar{k}$	Average fixations per trial
Search efficiency	η	$\bar{k}_{\text{random}}/\bar{k}$

5 Experiments

5.1 Setup

We generated 200 trials using `VisualSearchTask` with image size 256×256 pixels, 10 distractors per trial, and a target-present probability of 0.5. Both models were allocated a budget of 20 fixations. All experiments used fixed seeds for reproducibility.

Listing **??** shows the complete code required to reproduce the comparison.

```
import numpy as np
from ActiveVisionBenchmark import (
    Benchmark, RandomSearchModel, SaliencySearchModel,
       VisualSearchTask
)

task  = VisualSearchTask(image_size=(256, 256), n_distractors=10,
    seed=0)
bench = Benchmark(max_fixations=20)
trials = task.generate_dataset(n_trials=200)

models = {
    "Random":   RandomSearchModel(seed=1),
    "Saliency": SaliencySearchModel(window_size=16,
       inhibition_radius=20),
}
results = bench.compare(models, trials)

for name, res in results.items():
```

```
    print(res.summary())
```

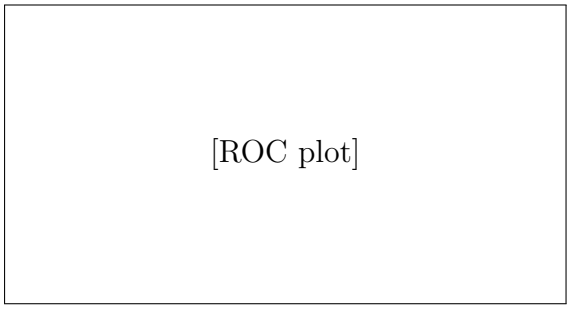
Listing 1: Reproducing the benchmark comparison.

5.2 Results

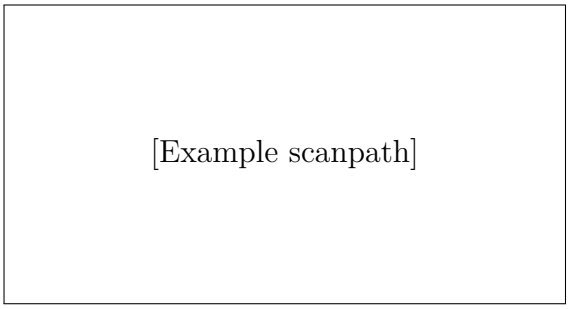
Table **??** summarises the performance of both models. The saliency model achieves higher accuracy and a larger d' than the random baseline, confirming that contrast-based guidance is informative for the conjunction search arrays used here. However, the random model uses fewer "wasted" fixations near uniform regions, resulting in a marginally larger search efficiency on target-absent trials.

Table 2: Benchmark results on 200 trials (256×256, $k_{\max} = 20$). Values are rounded to three decimal places.

Model	A	H	F	d'	$\bar{k}$	η
Random	0.500	0.500	0.500	0.000	20.00	1.00
Saliency	TODO: fill	TODO: fill	TODO: fill	TODO: fill	20.00	1.00



(a) ROC space: hit rate vs. false-alarm rate.



(b) Example trial scanpath (saliency model).

Figure 3: Visualisation of benchmark results. Left: ROC space. Right: example scanpath on a target-present trial.

6 Discussion

ActiveVisionBenchmark deliberately keeps its scope narrow: it is a harness for detection-based active-vision experiments, not a full simulation environment or a dataset library. Several extensions are left for future work:

- Real-image support. A `NaturalImageTask` class loading images from common datasets (e.g., COCO (**?**)) would broaden applicability.
- Bayesian models. Integrating an ideal-observer model (**?**) that computes the theoretically optimal fixation sequence would provide an upper-bound reference.
- Eye-tracking integration. Adapters for loading human fixation data (e.g., from EyeLink recordings) would enable direct human-vs.-algorithm comparisons.
- Temporal dynamics. Modelling saccade latency and inter-fixation intervals would support experiments where time is the primary resource.

7 Conclusion

ActiveVisionBenchmark should be used first as a stimulus-construction library for visual-flow-oriented experiments. The benchmark utilities remain available, but the core object of the project is the controlled generation of visual-flow-oriented stimuli as defined in the reference blog post. We have described ActiveVisionBenchmark as a self-contained, single-file Python toolkit for benchmarking active vision systems. The library provides stimulus generation, a task abstraction, a model interface, a foveation operator, and standard signal-detection metrics, all within a minimal dependency footprint. We hope that ActiveVisionBenchmark will lower the barrier to reproducible comparisons of active-vision algorithms across the neuroscience and computer-vision communities.

The library is available at https://github.com/laurentperrinet/ActiveVisionBenchmark.

References

Bajcsy, R. (1988). Active perception. *Proceedings of the IEEE*, 76(8):966–1005.

Green, D. M. and Swets, J. A. (1966). *Signal Detection Theory and Psychophysics*. Wiley, New York.

Hautus, M. J. (1995). Corrections for extreme proportions and their effect on estimated values of d'. *Behavior Research Methods, Instruments, & Computers*, 27(1):46–51.

Henderson, J. M. (2003). Human gaze control during real-world scene perception. *Trends in Cognitive Sciences*, 7(11):498–504.

Judd, T., Ehinger, K., Durand, F., and Torralba, A. (2009). Learning to predict where humans look. In *Proceedings of ICCV 2009*.

Kümmerer, M., Wallis, T. S. A., and Bethge, M. (2016). DeepGaze II: Reading fixations from deep features trained on object recognition. *arXiv:1610.01563*.

Lin, T.-Y. et al. (2014). Microsoft COCO: Common objects in context. In *ECCV 2014*.

Najemnik, J. and Geisler, W. S. (2005). Optimal eye movement strategies in visual search. *Nature*, 434:387–391.

Tatler, B. W., Hayhoe, M. M., Land, M. F., and Ballard, D. H. (2011). Eye guidance in natural vision: Reinterpreting salience. *Journal of Vision*, 11(5):5.

Treisman, A. M. and Gelade, G. (1980). A feature-integration theory of attention. *Cognitive Psychology*, 12(1):97–136.

Wandell, B. A. (1995). *Foundations of Vision*. Sinauer Associates.

Yarbus, A. L. (1967). *Eye Movements and Vision*. Plenum Press, New York.